Is it Worth Making Long Recipes?

Name(s): Dylan Pham, Andrew Lezaja

Website Link: https://dph4m.github.io/recipe_analysis_project/

In [1]:
```
import pandas as pd
import numpy as np

import plotly.express as px
pd.options.plotting.backend = 'plotly'

from lec_utils import * # Feel free to uncomment and use this. It'll make your
plotly graphs look like ours in lecture!
```

Step 1: Introduction

In [2]:
```
# TODO
print('''Why did we select the dataset?
Being college students both living in an overpriced apartment complex surrounded
by unaffordable restaurants in downtown Ann Arbor, the ability to cook good food
may sometimes seem like a life or death skill. We're always looking for ways to
improve our meals, making R&R the obvious choice. Beyond its immediate appeal, R&R
contains a large amount of diverse data, making it easy to ask interesting
questions about its information and food in general. The split between the recipe
and ratings tables will hopefully allow us to explore not only patterns within
recipes/food, but its relation and impact on people through ratings. There are a
multitude of columns, supplemented by large descriptions, recipes, reviews, etc.
R&R's depth and size will make for very interesting research and correlations to
be drawn. Specifically, we will be analyzing tags, description, and n_steps.
''')
```

```
Why did we select the dataset?
Being college students both living in an overpriced apartment complex surrounded by
unaffordable restaurants in downtown Ann Arbor, the ability to cook good food may so
metimes seem like a life or death skill. We're always looking for ways to improve ou
r meals, making R&R the obvious choice. Beyond its immediate appeal, R&R contains a
large amount of diverse data, making it easy to ask interesting questions about its
information and food in general. The split between the recipe and ratings tables wil
l hopefully allow us to explore not only patterns within recipes/food, but its relat
ion and impact on people through ratings. There are a multitude of columns, suppleme
nted by large descriptions, recipes, reviews, etc. R&R's depth and size will make fo
r very interesting research and correlations to be drawn. Specifically, we will be a
nalyzing tags, description, and n_steps.
```

Step 2: Data Cleaning and Exploratory Data Analysis

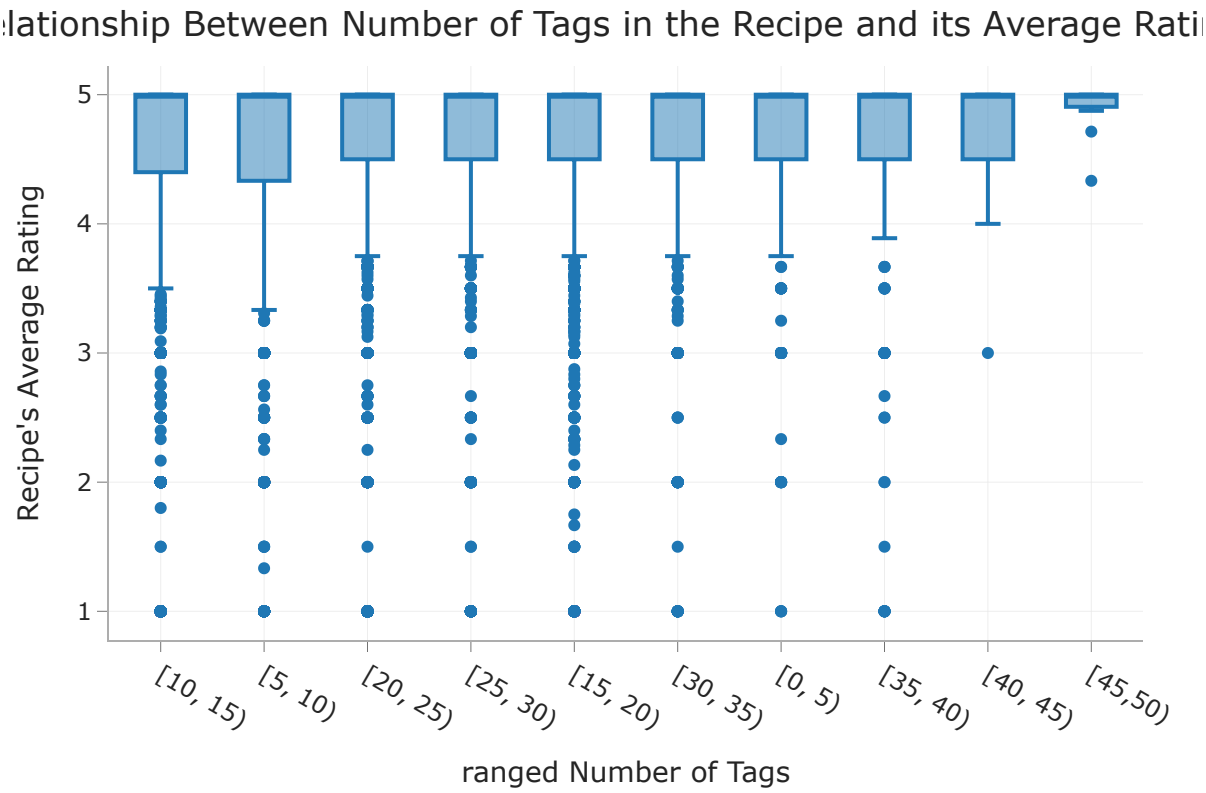
In [3]:
```
# TODO
recipes = pd.read_csv('RAW_recipes.csv')
interactions = pd.read_csv('RAW_interactions.csv')
print(len(recipes))
```

```
83782
```

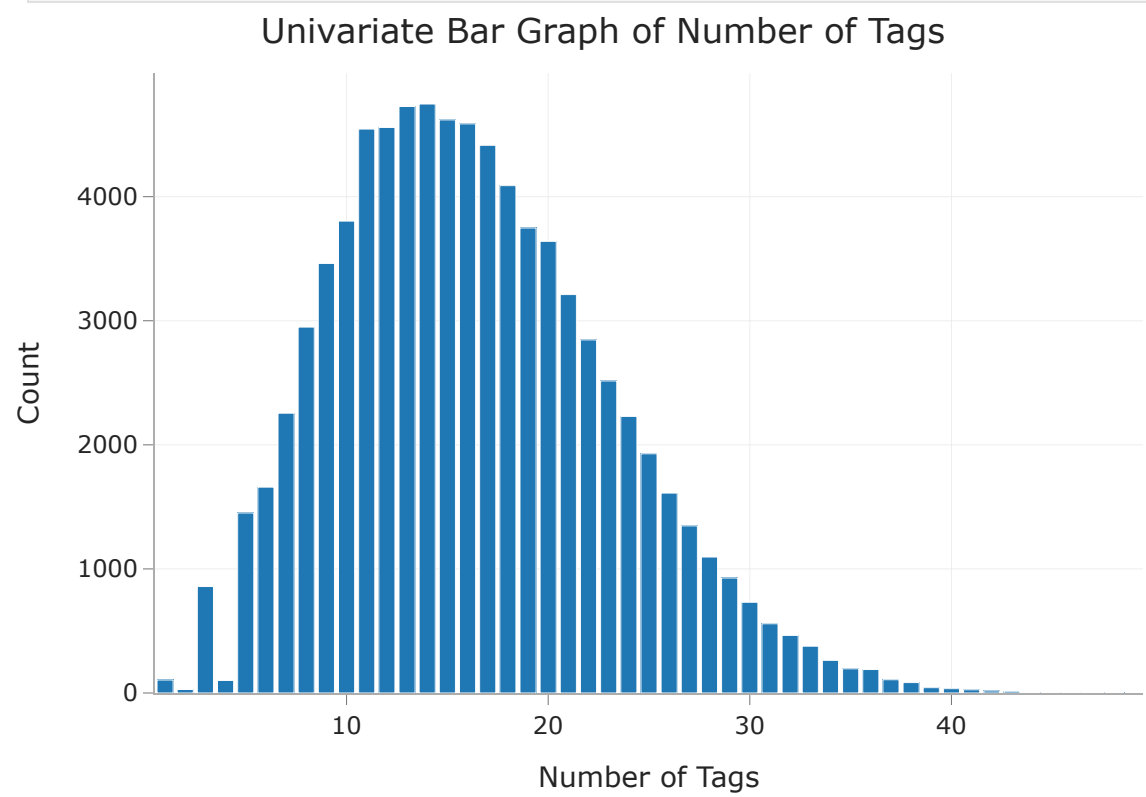
In [4]:
```
merged = recipes.merge(interactions, left_on = 'id',
                       right_on = 'recipe_id', how = 'left')
merged['rating'] = merged['rating'].replace(0, np.nan)
series = merged.groupby('id')['rating'].mean()
print(series)
recipes.set_index('id', inplace=True)
recipes['avgRatingPerRecipe'] = series
recipes.dropna(subset=['avgRatingPerRecipe'], inplace=True)
print(recipes['avgRatingPerRecipe'])
```

```
id
275022    3.0
275024    3.0
275026    3.0
         ...
537543    NaN
537671    NaN
537716    5.0
Name: rating, Length: 83782, dtype: float64
id
333281    4.0
453467    5.0
306168    5.0
         ...
308080    5.0
298512    1.0
298509    3.0
Name: avgRatingPerRecipe, Length: 81173, dtype: float64
```

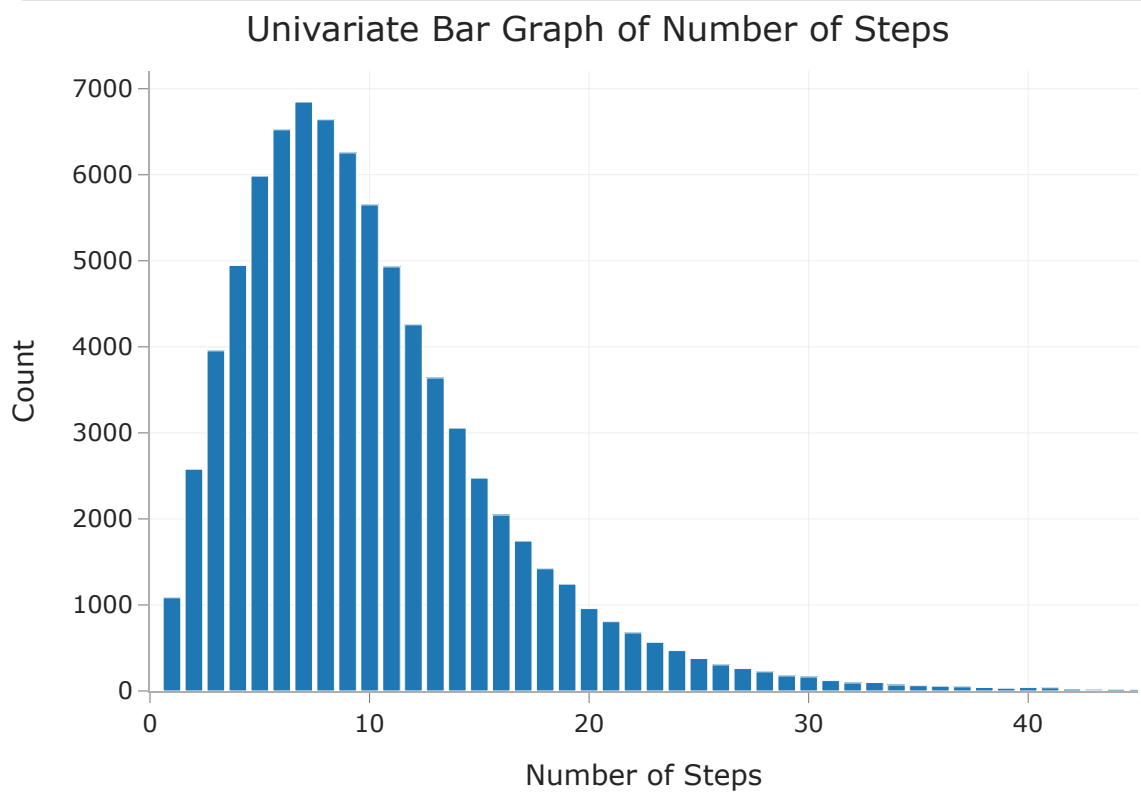
In [5]:
```
# TODO
#tags ONE HOT ENCODED
#n steps

#avg ratings = y axis
recipes['n_tags'] = recipes['tags'].str.count(",") + 1
bins = [0,5,10,15,20,25,30,35,40,45,50]
labels = ['[0, 5)', '[5, 10)', '[10, 15)', '[15, 20)', '[20, 25)',
          '[25, 30)', '[30, 35)', '[35, 40)', '[40, 45)', '[45,50)']
recipes['ranged Number of Tags'] = pd.cut(recipes['n_tags'],
                                          bins=bins, labels=labels, right=False)
fig = px.box(recipes, x='ranged Number of Tags', y='avgRatingPerRecipe',
                 labels={
                     "n_tags": "Number of Tags Recipe Contains",
                     "avgRatingPerRecipe": "Recipe's Average Rating",
                 },
                 title='Relationship Between Number of Tags in the Recipe and its
Average Rating')
# print(merged.shape)
fig.show()
```



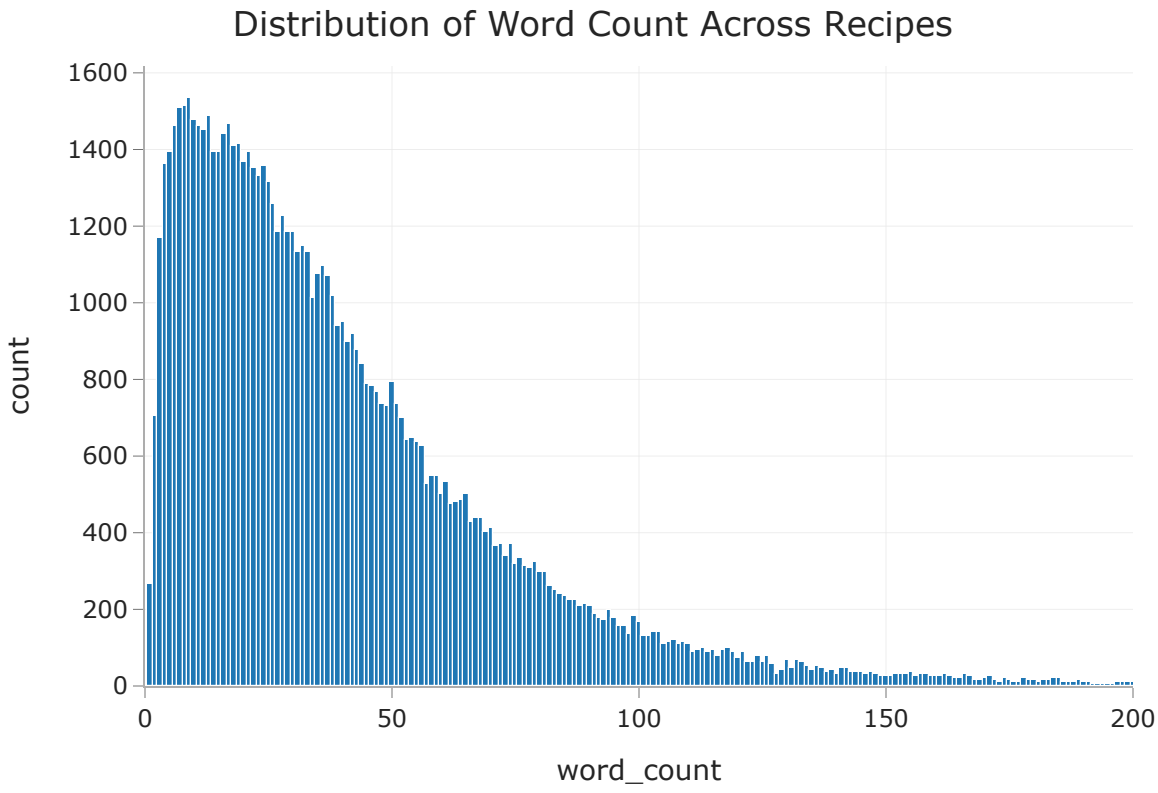
In [6]:
```
category_counts = recipes['n_tags'].value_counts().reset_index()
category_counts.columns = ['Number of Tags', 'Count']
fig2 = px.bar(category_counts, x='Number of Tags', y='Count',
              title='Univariate Bar Graph of Number of Tags')
fig2.show()
```



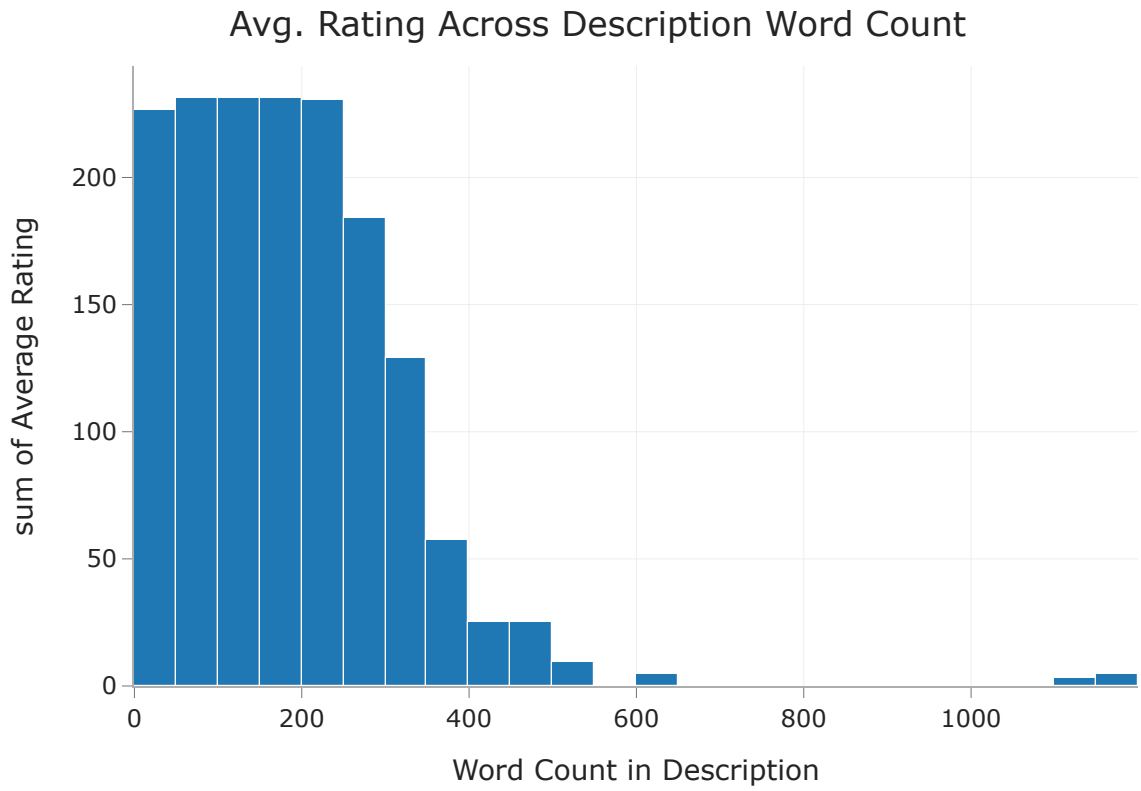
In [7]:
```
category_counts2 = recipes['n_steps'].value_counts().reset_index()
category_counts2.columns = ['Number of Steps', 'Count']
fig3 = px.bar(category_counts2, x='Number of Steps', y='Count',
                title='Univariate Bar Graph of Number of Steps')
fig3.update_layout(
    xaxis_range=[0, 45]  # Set the desired range for the x-axis
)
fig3.show()
fig3.write_html('univariate_n_steps.html', include_plotlyjs='cdn')
```



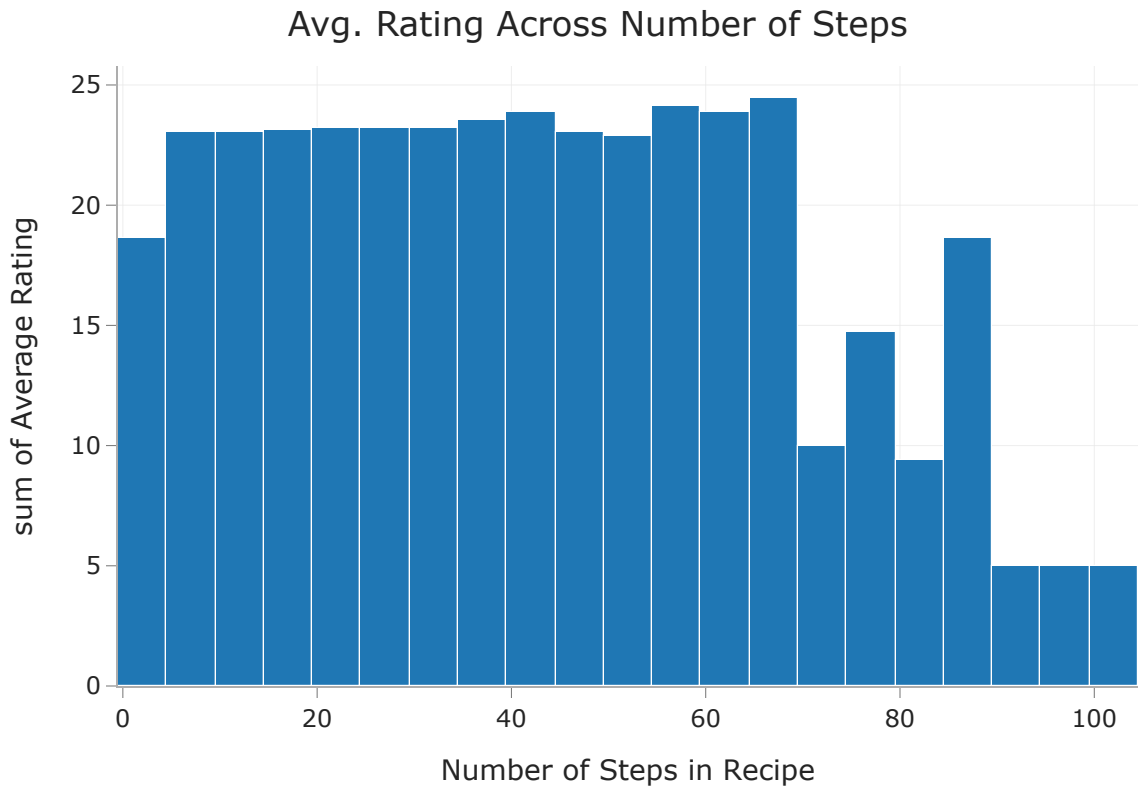
In [8]:
```
recipes['word_count'] = recipes['description'].str.split().str.len()
fig4 = px.histogram(recipes, x='word_count',
                    title='Distribution of Word Count Across Recipes')
fig4.update_layout(
    xaxis_range=[0, 200]  # Set the desired range for the x-axis
)
fig4.show()
```



In [9]:
```
recipes['desc_word_count'] = recipes['description'].str.split().str.len()
avg_rating_desc = recipes.groupby('desc_word_count')
['avgRatingPerRecipe'].mean().reset_index()
fig5 = px.histogram(avg_rating_desc, x='desc_word_count', y='avgRatingPerRecipe',
              title='Avg. Rating Across Description Word Count',
              labels={'desc_word_count': 'Word Count in Description',
                      'avgRatingPerRecipe': 'Average Rating'})
fig5.show()
```



In [10]:
```
avg_rating_steps = recipes.groupby('n_steps')
['avgRatingPerRecipe'].mean().reset_index()
fig6 = px.histogram(avg_rating_steps, x='n_steps', y='avgRatingPerRecipe',
              title='Avg. Rating Across Number of Steps',
              labels={'n_steps': 'Number of Steps in Recipe',
'avgRatingPerRecipe': 'Average Rating'}, nbins = 22)
fig6.show()
fig6.write_html('rate_across_n_steps.html', include_plotlyjs='cdn')
```



In [11]:
```
!pip install tabulate
recipes = pd.DataFrame(recipes)
print(recipes.columns)
print(recipes[['name','avgRatingPerRecipe', 'n_steps', 'n_tags',
'desc_word_count']].head(20).to_markdown(index=False))
```

```
Requirement already satisfied: tabulate in /home/dpham/miniforge3/envs/pds/lib/pytho
n3.10/site-packages (0.9.0)
Index(['name', 'minutes', 'contributor_id', 'submitted', 'tags', 'nutrition',
       'n_steps', 'steps', 'description', 'ingredients', 'n_ingredients',
       'avgRatingPerRecipe', 'n_tags', 'ranged Number of Tags', 'word_count',
       'desc_word_count'],
      dtype='object')
| name                                    |   avgRatingPerRecipe |   n_steps |   n_t
ags |   desc_word_count |
|:----------------------------------------|----------------------:|----------:|------
---:|------------------:|
| 1 brownies in the world    best ever    |                    4 |        10 |
14 |                41 |
| 1 in canada chocolate chip cookies      |                    5 |        12 |
9 |                42 |
| 412 broccoli casserole                  |                    5 |         6 |
10 |                64 |
| millionaire pound cake                  |                    5 |         7 |
20 |                34 |
| 2000 meatloaf                           |                    5 |        17 |
10 |                29 |
| 5 tacos                                 |                    4 |         5 |
26 |                 5 |
| 50 chili   for the crockpot             |                    5 |         4 |
22 |               125 |
| blepandekager   danish   apple pancakes |                    5 |        10 |
10 |                23 |
| lplermagrone                            |                    5 |        10 |
11 |               108 |
| lplermagrone  herdsman s macaroni       |                    5 |        14 |
13 |                99 |
| rter med flsk   pea soup with pork      |                    5 |         5 |
23 |                29 |
| rtsoppa  swedish yellow pea soup        |                    4 |        14 |
11 |                46 |
| pinards en branche  sauted spinach      |                    5 |        13 |
24 |                 5 |
| go to bbq sauce for ribs                |                    5 |         3 |
11 |                46 |
| bbq spray recipe    it really works     |                 4.75 |         5 |
18 |                32 |
| berry french toast  oatmeal             |                 4.75 |         5 |
12 |                32 |
| big easy   gumbo                        |                  4.5 |        11 |
28 |                42 |
| burek  or feta cheese  phyllo pie       |                  4.5 |        38 |
12 |               101 |
| california roll   salad                 |                    5 |         8 |
5 |                55 |
| cheeeezy  potatoes                      |                    5 |         8 |
22 |                 8 |
```

Step 3: Framing a Prediction Problem

In [12]:
```
# TODO
print('''
What is the column you plan on trying to predict in Part 1, Steps 3-5? Is it a
classification or regression problem? Explain.

For our model, we plan on addressing a regression problem predicting the average
rating (column) of a recipe given multiple features: tag length, description word
length, and the number of recipe steps. We will be continuing the theme of
analyzing recipe “size” in terms of the aforementioned variables, and using them
to draw conclusions on existing recipes from the data. We are brainstorming using
methods like feature standardization to extract more insightful information out of
coefficients, using MLR to combine multiple features, using regularization and
gradient descent to improve testing performance, and tuning hyperparameters to
build an efficient model. Even though ratings can only be entered discreetly (1-
5), we will be predicting avg. ratings and so this is a regression problem through
and through.
''')
```

```
What is the column you plan on trying to predict in Part 1, Steps 3-5? Is it a class
ification or regression problem? Explain.

For our model, we plan on addressing a regression problem predicting the average rat
ing (column) of a recipe given multiple features: tag length, description word lengt
h, and the number of recipe steps. We will be continuing the theme of analyzing reci
pe “size” in terms of the aforementioned variables, and using them to draw conclusio
ns on existing recipes from the data. We are brainstorming using methods like featur
e standardization to extract more insightful information out of coefficients, using
MLR to combine multiple features, using regularization and gradient descent to impro
ve testing performance, and tuning hyperparameters to build an efficient model. Even
though ratings can only be entered discreetly (1-5), we will be predicting avg. rati
ngs and so this is a regression problem through and through.
```

Step 4: Baseline Model

In [13]:
```
# TODO
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.naive_bayes import GaussianNB
from sklearn.neural_network import MLPRegressor
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline, FunctionTransformer

def unfits():
    tree = DecisionTreeRegressor(max_depth=4, random_state=98)
    forest = RandomForestRegressor(max_depth=4, random_state=98)
    knn = KNeighborsRegressor(n_neighbors=3)
    gaussian = GaussianNB()
    logReg = LogisticRegression(C=np.inf, random_state=98)
    neural = MLPRegressor(random_state=98)
```

```
    linReg = LinearRegression()

    dict = {
        'Decision Tree (max_depth=4)' : tree,
        'Random Forest (max_depth=4)' : forest,
        'KNN (k=10)' : knn,
        'Naive Bayes' : gaussian,
        'Logistic Regression' : logReg,
        'Neural Network' : neural,
        'Linear Regression' : linReg
    }
    return dict

def train_standardized_models(X_train, y_train):

    dict = unfits()
    ret = {}
    X_train2 = recipes[['n_tags', 'desc_word_count', 'n_steps']]
    for key, value in dict.items():
        pl = make_pipeline(
            StandardScaler(),
            value
        )
        pl.fit(X_train2, y_train)
        ret[key] = pl
    return ret

features = ['n_steps', 'n_tags', 'desc_word_count']
tar = 'avgRatingPerRecipe'
data = recipes[features + [tar]].dropna()
X = data[features]
y = data[tar]
X_train, X_test, y_train, y_test =
train_test_split(X,y,test_size=0.2,random_state=98)

models = unfits()
pipe = make_pipeline(
    StandardScaler(),
    models['KNN (k=10)']
)

pipe.fit(X_train, y_train)
y_train_pred = pipe.predict(X_train)
y_test_pred = pipe.predict(X_test)

train_mse = mean_squared_error(y_train, y_train_pred)
test_mse = mean_squared_error(y_test, y_test_pred)
# print(train_mse)
# print(test_mse)

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import ast
```

```python
def count_words(text):
    return len(str(text).split())

def parse_nutrition(nutrition_str):
    try:
        nutrition_list = ast.literal_eval(nutrition_str)
        return pd.Series({
            'calories': nutrition_list[0],
            'total_fat_pdv': nutrition_list[1],
            'sugar_pdv': nutrition_list[2],
            'sodium_pdv': nutrition_list[3],
            'protein_pdv': nutrition_list[4],
            'saturated_fat_pdv': nutrition_list[5],
            'carbohydrates_pdv': nutrition_list[6]
        })
    except (ValueError, SyntaxError):
        return pd.Series({
            'calories': np.nan,
            'total_fat_pdv': np.nan,
            'sugar_pdv': np.nan,
            'sodium_pdv': np.nan,
            'protein_pdv': np.nan,
            'saturated_fat_pdv': np.nan,
            'carbohydrates_pdv': np.nan
        })

# Parse the nutrition column into individual features
nutrition_df = recipes['nutrition'].apply(ast.literal_eval).apply(pd.Series)
nutrition_df.columns = ['calories', 'total_fat', 'sugar', 'sodium', 'protein',
'sat_fat', 'carbs']
recipes = pd.concat([recipes, nutrition_df], axis=1)

# Drop rows with missing values in any feature or target column
features = ['n_steps', 'desc_word_count', 'n_tags', 'calories', 'total_fat',
'sugar', 'sodium', 'protein', 'sat_fat', 'carbs']
recipes = recipes.dropna(subset=features + ['avgRatingPerRecipe'])

# Split into X and y
X = recipes[features]
y = recipes['avgRatingPerRecipe']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

# Define the pipeline
baseline_pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LinearRegression())
])

# Train the model
baseline_pipe.fit(X_train, y_train)

# Predict and evaluate
y_pred = baseline_pipe.predict(X_test)
```

```
mse = mean_squared_error(y_test, y_pred)
print("Baseline Mean Squared Error (MSE):", mse)
```

```
Baseline Mean Squared Error (MSE): 0.4102268861969451
```

Step 5: Final Model

In [14]:

```
# TODO

from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV, cross_val_score
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import QuantileTransformer

# Ensure problematic columns are numeric
recipes['protein'] = recipes['protein'].astype(float)
recipes['calories'] = recipes['calories'].astype(float)
recipes['desc_word_count'] = recipes['desc_word_count'].astype(float)
recipes['n_steps'] = recipes['n_steps'].astype(float)

# Feature Engineering
recipes['desc_per_step'] = recipes['desc_word_count'] / recipes['n_steps']
recipes['protein_per_calorie'] = recipes['protein'] /
recipes['calories'].replace(0, np.nan)
recipes['sugar_to_fat'] = recipes['sugar'] / recipes['total_fat'].replace(0,
np.nan)
recipes['fat_to_calorie'] = recipes['total_fat'] / recipes['calories'].replace(0,
np.nan)

# Define final features
base_features = ['n_steps', 'desc_word_count', 'n_tags']
nutrition_features = ['calories', 'total_fat',
                      'sugar', 'sodium', 'protein', 'sat_fat', 'carbs']
engineered_features = ['desc_per_step', 'protein_per_calorie',
                       'sugar_to_fat', 'fat_to_calorie']
all_features = base_features + nutrition_features + engineered_features

# Drop any rows with missing values
recipes = recipes.dropna(subset=all_features + ['avgRatingPerRecipe'])

# Split data
X = recipes[all_features]
y = recipes['avgRatingPerRecipe']
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

# Define preprocessing
quantile_features = ['calories', 'total_fat', 'sugar', 'sat_fat']
standard_features = list(set(all_features) - set(quantile_features))

preprocessor = ColumnTransformer([
    ('quantile', QuantileTransformer(output_distribution='normal'),
quantile_features),
    ('standard', StandardScaler(), standard_features)
])
```

```python
# Define model pipeline
final_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('regressor', RandomForestRegressor(random_state=42))
])

# Define hyperparameters
param_grid = {
    'regressor__n_estimators': [50, 100],
    'regressor__max_depth': [5, 10, None],
    'regressor__min_samples_split': [2, 5]
}

# Grid search
grid_search = GridSearchCV(final_pipeline, param_grid,
                           cv=5, scoring='neg_mean_squared_error', n_jobs=-1)
grid_search.fit(X_train, y_train)

# Best model
best_model = grid_search.best_estimator_

# Evaluate
train_preds = best_model.predict(X_train)
test_preds = best_model.predict(X_test)
train_mse = mean_squared_error(y_train, train_preds)
test_mse = mean_squared_error(y_test, test_preds)
cv_scores = cross_val_score(best_model, X_train,
                            y_train, cv=5, scoring='neg_mean_squared_error')

print("Final Model Results:")
print("Best Params:", grid_search.best_params_)
print("Train MSE:", train_mse)
print("Test MSE:", test_mse)
print("Cross-validation MSEs:", -cv_scores)
print("CV MSE (mean ± std): {:.4f} ± {:.4f}".format(-cv_scores.mean(),
cv_scores.std()))
```

```
Final Model Results:
Best Params: {'regressor__max_depth': 5, 'regressor__min_samples_split': 5, 'regress
or__n_estimators': 100}
Train MSE: 0.41309462385417184
Test MSE: 0.3956002249220862
Cross-validation MSEs: [0.4  0.42 0.43 0.42 0.42]
CV MSE (mean ± std): 0.4177 ± 0.0107
```